We sample these quantities as

$$\mathbf{s} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}/d) \ ,$$
$$\eta \sim \mathcal{N}(\mathbf{0}, \sigma\mathbf{I}/d) \ .$$

A training set is sampled such that half the training examples belong to class 1, and half belong to class 0. Crucially, the training set consists of L fixed symbols $\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_L$ sampled prior to the experiment. All training examples are constructed from these L symbols. During testing, symbols are sampled afresh, forcing the model to generalize. If the noise variance σ is not explicitly stated, then we take it to be $\sigma = 0$. We use a base learning rate $\alpha_0 = 0.1$ with batches of size 128.

G.3 PSVRT

The PSVRT task consists of a single-channel square image with two blocks of bit-patterns. If the bit-patterns match exactly, then the image belongs to the *same* class. If the bit-patterns differ, then the image belongs to the *different* class. Images are flattened before being passed to the MLP.

Images are patch-aligned to prevent overlapping bit-patterns. An image is tiled by non-overlapping square regions which may be filled by bit-patterns. No two bit-patterns may share a single patch. Unless otherwise stated, we use patches that are 5 pixels to a side, and images that are 5 patches to a side, for a total of 25 by 25 pixels.

One important feature of PSVRT is that the inputs do not grow in norm as their dimension increases. Because there are only ever two patches in an image, regardless of its size, the total norm of the input remains constant regardless of the image dimensions. As a result, the $1/\sqrt{d}$ scaling on the MLP output is extraneous for PSVRT, and we remove it in these experiments.

A subset of all possible bit-patterns are used for training. The remaining unseen bit-patterns are used for testing. We use a base learning rate $\alpha_0 = 0.5$ with batches of size 128.

G.4 Pentomino

The Pentomino task consists of a single-channel square image with two pentomino shapes. If the shapes are the same (up to rotation, but not reflection), then the image belongs to the *same* class. If the bit-patterns differ, then the image belongs to the *different* class. Images are flattened before being passed to the MLP.

Like before, images are patch-aligned. To provide a border around each pentomino, patches are 7 pixels to a side. Unless otherwise stated, images are 2 patches to a side, for a total of 14 by 14 pixels.

As with PSVRT, the inputs for Pentomino do not grow in norm as their dimension icnreases. There are only ever two pentomino shapes in an image, regardless of its dimension, so the total norm of the input remains constant. Like with PSVRT, we remove the $1/\sqrt{d}$ output scaling on the MLP for these experiments.

There are a total of 18 possible pentomino shapes. A subset of these 18 is held out for testing, and the model trains on the remainder. To improve training stability, mild Gaussian blurs are randomly applied to training images, but not testing images. We use a base learning rate $\alpha_0 = 0.5$ with batches of size 128.

G.5 CIFAR-100

The CIFAR-100 same-different task consists of full-color images taken from the CIFAR-100 dataset. Images are 32 by 32 pixels, and depict 1 of among 100 different classes. To form an input example, we place two images side-by-side, forming a larger 64 by 32 pixel image. If the images come from the same class (but are not necessarily the same exact image), the example belongs to the *same* class. If the images come from different classes, the example belong to the *different* class.

To separate an MLP's ability to reason about equality from its ability to extract meaningful visual features, we first pass the image through a VGG-16 backbone pretrained on ImageNet. Activations are then taken from an intermediate layer, flattened, and passed to the MLP. Because VGG-16 activations are coordinate-wise $O(1)$ in magnitude, we normalize them by $1/\sqrt{d}$ before input to the model. The resulting performance of the MLP from activations of each layer are plotted in Figure G1.

Of the 100 total classes, a subset is held out for testing, and the model trains on the remainder. We use a base learning rate $\alpha_0 = 0.01$ with batches of size 128.

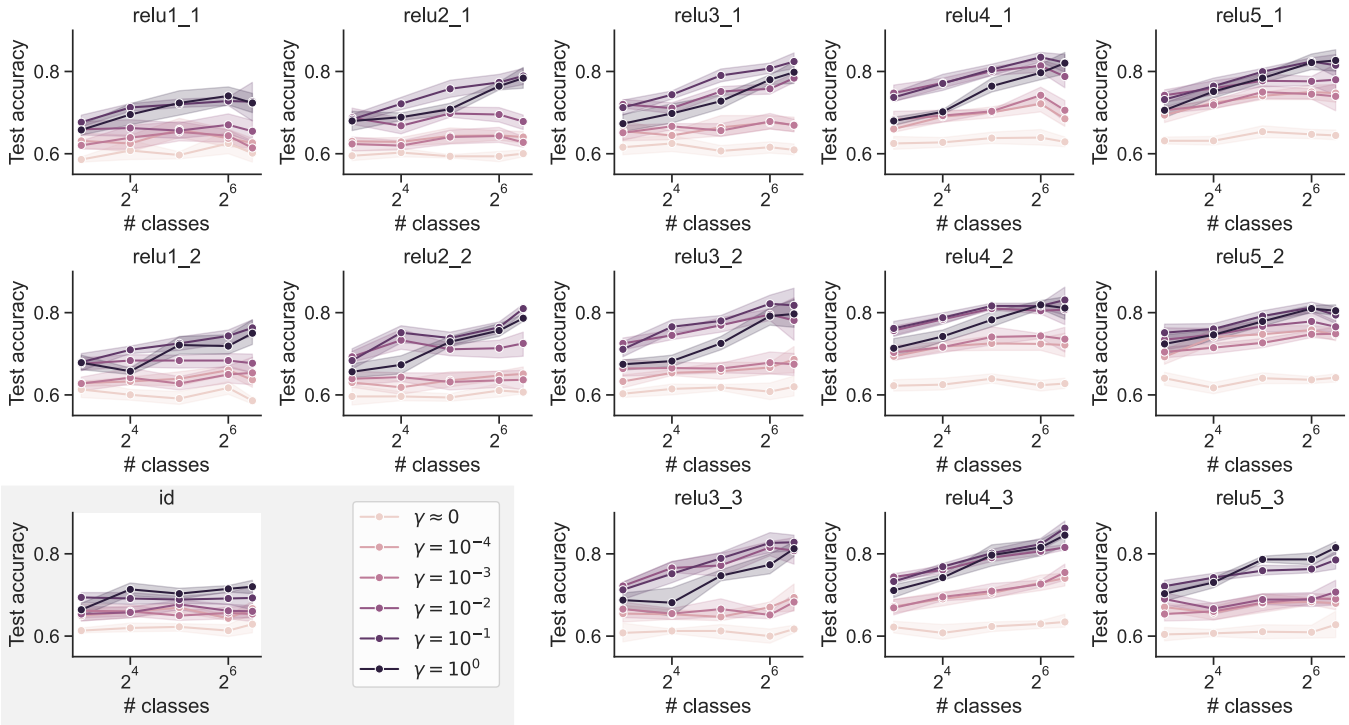

Figure G1: **CIFAR-100 same-different accuracy across different VGG-16 activations.** Activations are named by `relu[block]_[layer]` The plot with name `id` corresponds to using the raw images directly without first preprocessing in VGG-16. Earlier and later layers demonstrate an interesting collapse where learning richness does not seem to impact classification accuracy very strongly. Intermediate layers suggest that greater learning richness tends to perform better, though the richest model tends to do poorly. Shaded error regions correspond to 95 percent confidence intervals estimated from 6 runs.